

L10: Prompting & Parameter Efficient Fine-Tuning (PEFT)

FICT, UTAR

Table of Contents

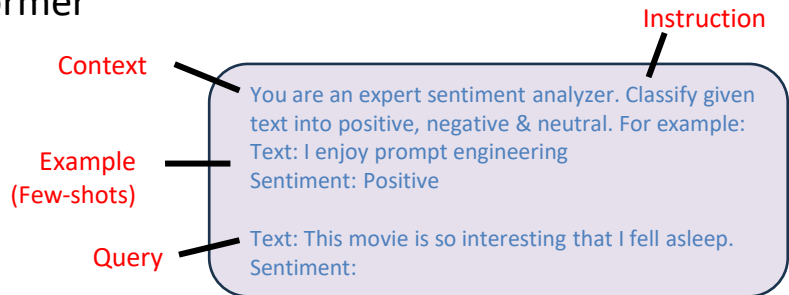
1. Prompting
 - Few-shot Prompting
2. Parameter Efficient Finetuning (PEFT)
 - Prompt Tuning
 - LoRA

Prompting

Few-shot Prompting

- LLM is a general task performer

Prompt engineering is the **art** and **science** of **crafting inputs** to an LLM model. A good prompt guides the model to provide the correct answer.



- GPT-3 delivered SOTA performance on some tasks with **few-shot prompting** (no fine-tuning)

Setting	LAMBADA (acc)	LAMBADA (ppl)	StoryCloze (acc)	HellaSwag (acc)
SOTA	68.0 ^a	8.63 ^b	91.8^c	85.6^d
GPT-3 Zero-Shot	76.2	3.00	83.2	78.9
GPT-3 One-Shot	72.5	3.35	84.7	78.1
GPT-3 Few-Shot	86.4	1.92	87.7	79.3

[Brown, T., et al., [Language models are few-shot learners](#). NeurIPS 2020]

Parameter Efficient Fine-Tuning (PEFT)

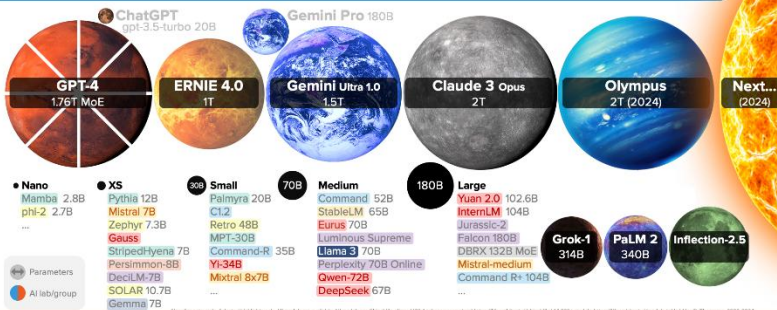
When to fine-tune?

- Start with **prompting** and **RAG**, and only finetune if you really need task/domain specialization or efficiency.
- When to finetune?
 - **Domain adaptation**: The data is very different from what the base model has seen
 - **Task specialization**: To achieve good performance on some narrow task
 - **Efficiency**: Finetune a smaller language model to outperform larger models on a specialized task

Why Parameter-Efficient Finetuning (PEFT)?

- Better performance than fine-tuning in the **low-data regime**.
- Generalize better in **out-of-domain** scenarios.
- Full fine-tuning is **computationally** intensive.
 - Out of memory (OOM) issues, slow speed

LARGE LANGUAGE MODEL HIGHLIGHTS (APR/2024)



LifeArchitect.ai/models

LLM with 7B (Estimation)

Model weights : $7B * 4 \text{ bytes} = 28GB$
Gradients : $7B * 4 \text{ bytes} = 28GB$
Adam : $2 * 28GB = 56GB$
Activation : 8.4GB to 14GB (estimated)
Total : $28GB + 28GB + 56GB + 14GB = 126GB$

LLM with 7B (16-bit quantization)

Model weights (F16) : $7B * 2 \text{ bytes} = 14GB$
Gradients (FP16/BF16) : $7B * 2 \text{ bytes} = 14GB$
Adam : $2 * 14GB = 28GB$
Activation : 4.2GB to 7GB (estimated)
Total : $14GB + 14GB + 28GB + 7GB = 63GB$

LLM with 7B (4-bit quantization)

Model weights (4-bit) : $7B * 4 \text{ bit} = 3.5GB$
Gradients (4 bit) : $7B * 4 \text{ bit} = 3.5GB$
Adam (4 bit) : $2 * 3.5GB = 7GB$
Activation : 1 to 2GB (estimated)
Total : $3.5GB + 3.5GB + 7GB + 2GB = 16GB$

Model size scales almost 2x of single GPU memory.

Previously, we cannot pre-train the model but now fine-tuning is a challenge!

Why Parameter-Efficient Finetuning (PEFT)?

- Full finetuning for multiple downstream tasks suffers from **storage** and **scalability** Issues - Different full-size model for each task

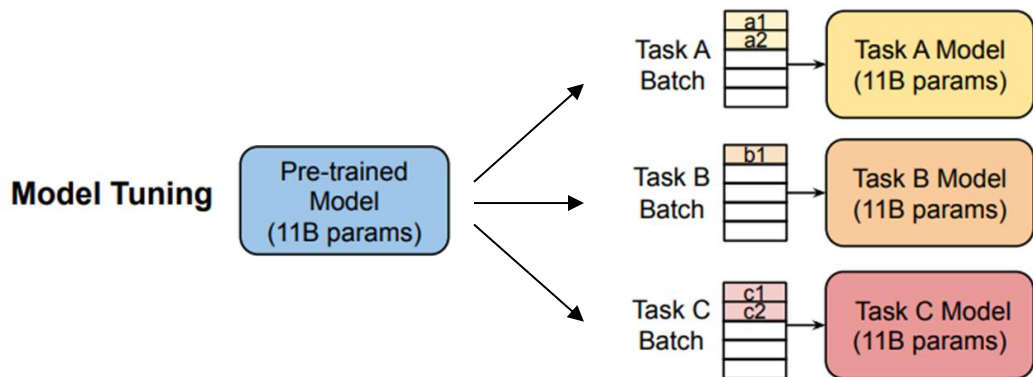
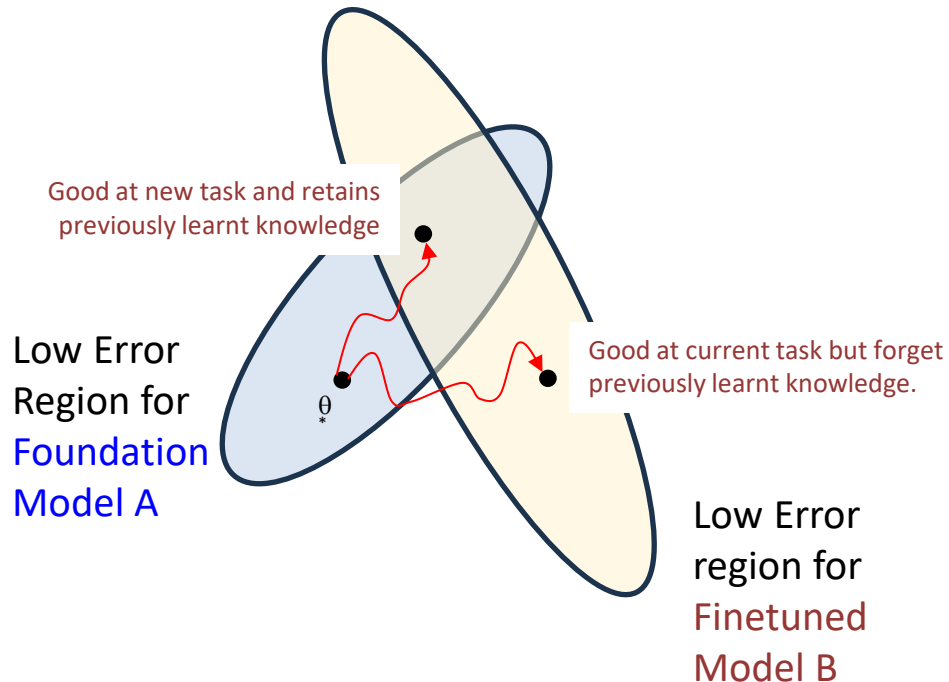


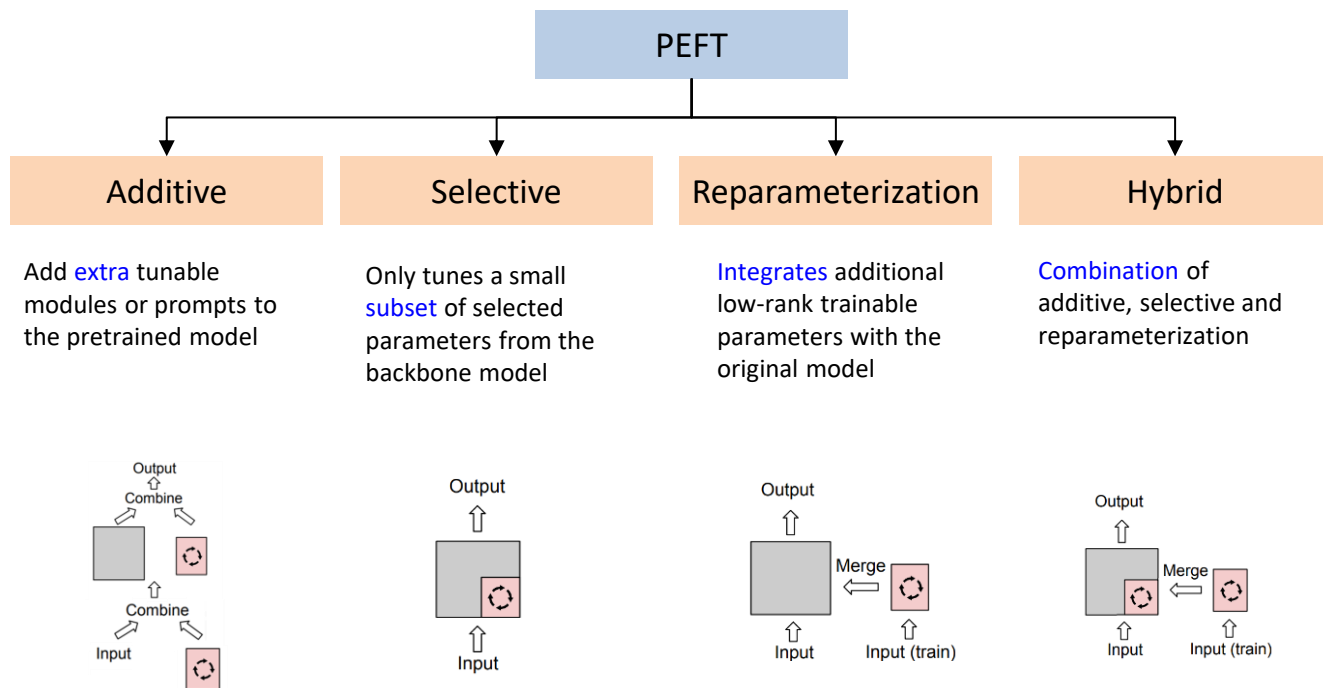
Fig: Scalability issues of finetuning LLMs

Why Parameter-Efficient Finetuning (PEFT)?

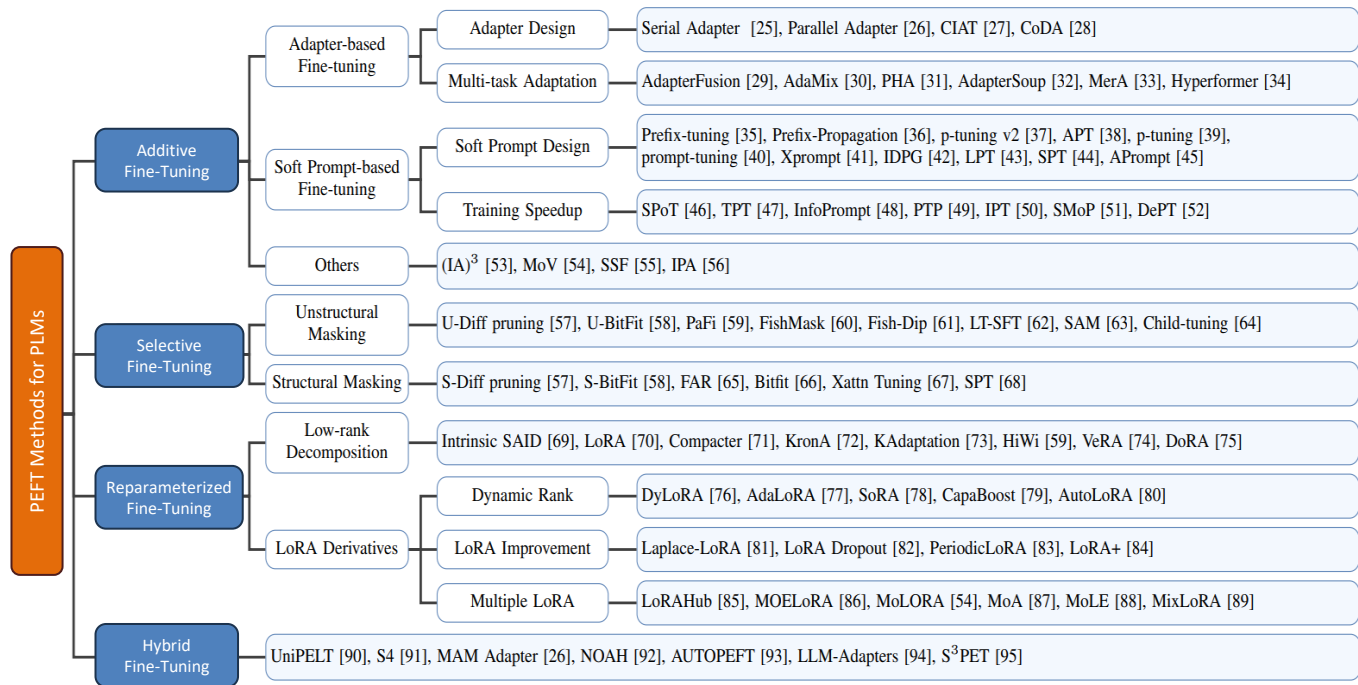
- Full fine-tuning is more prone to catastrophic **forgetting** (Loss of previously learned knowledge)



Types of PEFT



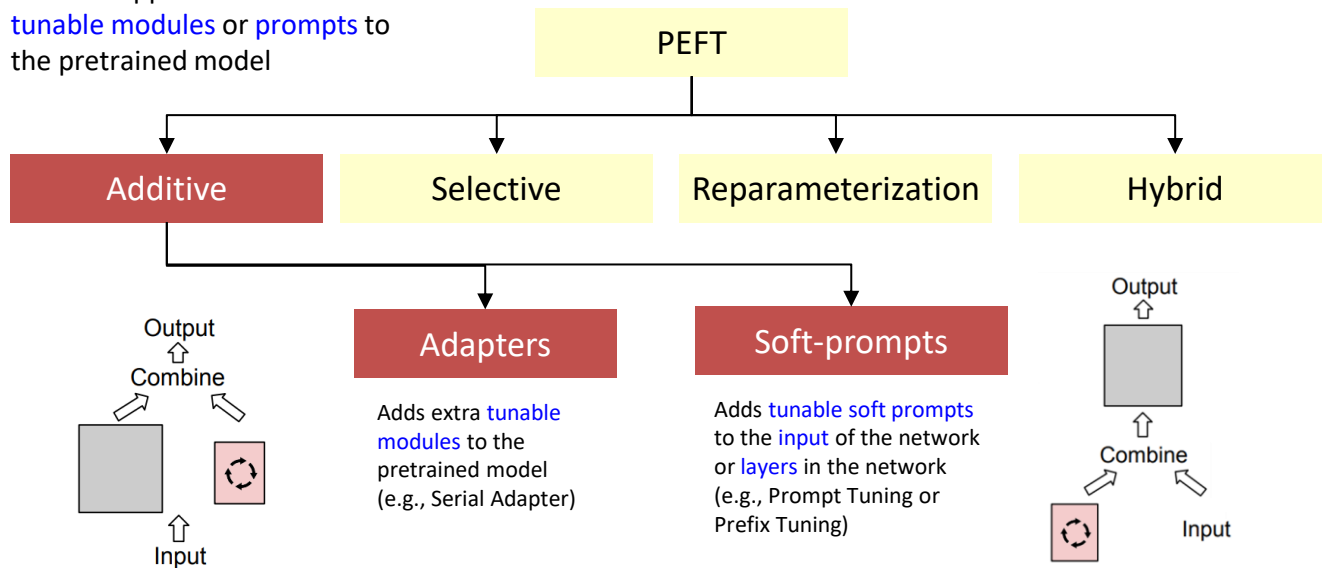
Taxonomy of PEFT



Parameter Efficient Fine-Tuning: Prompt Tuning Methods

Additive PEFT

Additive approaches adds **extra tunable modules** or **prompts** to the pretrained model



How you ask questions is important

- LLM is sensitive to how you ask your questions for the pretrained model and fine-tuned model.

Prompt	P@1 w/o PT
[X] is located in [Y]. (<i>original</i>)	31.3
[X] is located in which country or state? [Y].	19.8
[X] is located in which country? [Y].	31.4
[X] is located in which country? In [Y].	51.1

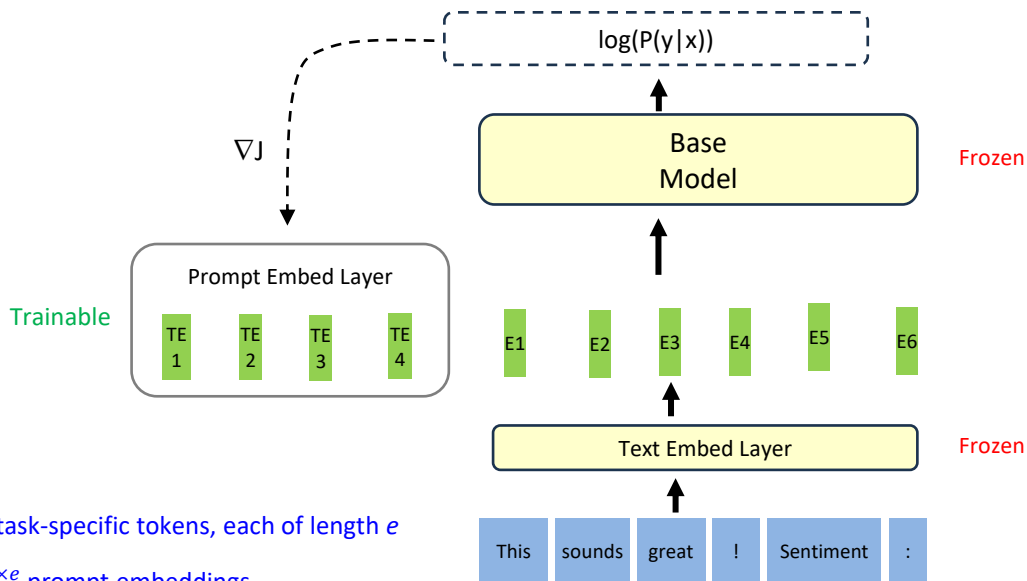
Precision@1 on LAMA-TREx P17 with a frozen BERT-base-cased for different template
(Liu, X., et al., [GPT understands, too](#). *AI Open.*, 2023)

Entity Template T^+	None-Entity Template T^-	Dev F1
$\langle \text{candidate_span} \rangle$ is a $\langle \text{entity_type} \rangle$ entity	$\langle \text{candidate_span} \rangle$ is not a named entity	95.27
The entity type of $\langle \text{candidate_span} \rangle$ is $\langle \text{entity_type} \rangle$	The entity type of $\langle \text{candidate_span} \rangle$ is none entity	95.15
$\langle \text{candidate_span} \rangle$ belongs to $\langle \text{entity_type} \rangle$ category	$\langle \text{candidate_span} \rangle$ belongs to none category	88.42
$\langle \text{candidate_span} \rangle$ should be tagged as $\langle \text{entity_type} \rangle$	$\langle \text{candidate_span} \rangle$ should tagged as none entity	76.80

Performance of BART for NER task after fine-tuning for different templates
(Leyang Cui, et al. [Template-Based Named Entity Recognition Using BART](#). *ACL-IJCNLP 2021*)

Prompt Tuning

- Prepend and learn k task-specific tuneable tokens to input text
- The base model is frozen.



- n_p learnt task-specific tokens, each of length e
- $P_e \in \mathbb{R}^{n_p \times e}$ prompt embeddings

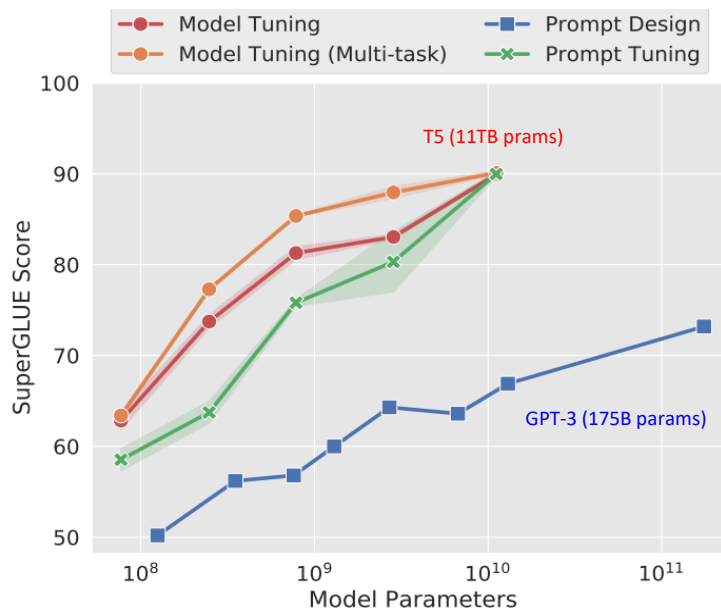
Prompt Tuning: Number of parameters used

- Huge savings in terms of trainable parameters (maximum 0.099% of total parameters). Bigger savings for larger models

T5 Size	Prompt Length	Trainable Parameters	Total Parameters	Percent Trainable
Small	1	512	76,961,664	0.00067%
	5	2,560	76,963,712	0.00333%
	20	10,420	76,971,572	0.01330%
	50	25,600	76,986,752	0.03325%
	100	51,200	77,012,352	0.06648%
	150	76,800	77,037,952	0.09969%
Base	1	768	247,578,624	0.00031%
	5	3,840	247,581,696	0.00155%
	20	15,360	247,593,216	0.00620%
	50	38,400	247,616,256	0.01551%
	100	76,800	247,654,656	0.03101%
	150	115,200	247,693,056	0.04651%
Large	1	1,024	783,151,104	0.00013%
	5	5,120	783,155,200	0.00065%
	20	20,480	783,170,560	0.00262%
	50	51,200	783,201,280	0.00654%
	100	102,400	783,252,480	0.01907%
	150	153,600	783,303,680	0.01961%
XL	1	2,048	2,849,759,232	0.00007%
	5	10,240	2,849,767,424	0.00036%
	20	40,960	2,849,798,144	0.00143%
	50	102,400	2,849,859,584	0.00359%
	100	204,800	2,849,961,984	0.00718%
	150	307,200	2,850,064,384	0.01078%
XXL	1	4,096	11,135,336,448	0.00004%
	5	20,480	11,135,352,832	0.00018%
	20	81,920	11,135,414,272	0.00074%
	50	204,800	11,137,380,352	0.00184%
	100	409,600	11,135,741,952	0.00368%
	150	614,400	11,135,946,752	0.00552%

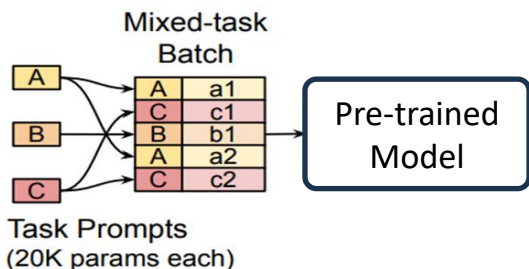
Prompt Tuning: Results

- Prompt tuning of T5 **matches** the quality of model tuning as the **model size increases**, while enabling the reuse of frozen model for all tasks.
- **Outperforms** few-shot **manual prompts** with GPT-3 in many evaluated tasks and model size



Prompt Tuning: Benefits

- **Fast and small** – only need to train and store a small task-specific prompt for each task. The pretrained model is **frozen** and shared by all tasks
- Modular design. Able to perform **mixed-task inference**
- **Domain adaptability** – Tends to give stronger zero-shot performance than finetuning especially on datasets with large domain shifts such as TextbookQA



Dataset	Domain	Model	Prompt	Δ
SQuAD	Wiki	94.9 $\pm$ 0.2	94.8 $\pm$ 0.1	-0.1
TextbookQA	Book	54.3 $\pm$ 3.7	66.8 $\pm$ 2.9	+12.5
BioASQ	Bio	77.9 $\pm$ 0.4	79.1 $\pm$ 0.3	+1.2
RACE	Exam	59.8 $\pm$ 0.6	60.7 $\pm$ 0.5	+0.9
RE	Wiki	88.4 $\pm$ 0.1	88.8 $\pm$ 0.2	+0.4
DuoRC	Movie	68.9 $\pm$ 0.7	67.7 $\pm$ 1.1	-1.2
DROP	Wiki	68.9 $\pm$ 1.7	67.1 $\pm$ 1.9	-1.8

Table: F1 performance for models trained on SQuAD and evaluated on out-of-domain datasets from MRQA 2019 tasks.

Parameter Efficient Fine-Tuning: LoRA

Low-Rank Adaptation (LoRA)

Solves two problems during fine-tuning:

1. How to make it more **efficient**?

Freeze as much parameter as possible and update as few as possible

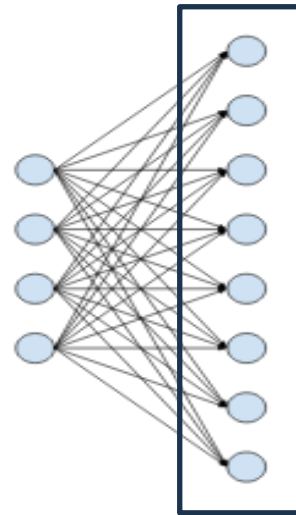
2. How to mitigate **forgetting catastrophe** after finetuning?
Avoid changing the original network parameters

Low-Rank Adaptation (LoRA)

- Assume that the LLM's parameters $\mathbf{W}$ has a **low intrinsic rank** (information in a matrix can be represented with fewer dimensions than the original matrix).
- Reparameterize $\mathbf{W}$ by adding an **offset** $\Delta\mathbf{W}$.

$$\mathbf{W}_{lora} = \mathbf{W} + \frac{\alpha}{r} \Delta\mathbf{W}$$

- If the weight matrix $\mathbf{W}$ has low intrinsic rank, then the update matrix $\Delta\mathbf{W}$ might also have low intrinsic rank too.



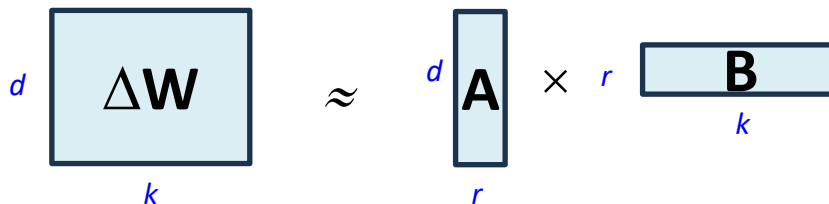
$$\mathbf{W} \in d \times k$$

Low-Rank Adaptation (LoRA)

- Factorize the the update matrix ΔW to two low-rank matrices A and B

$$W_{lora} = W + \frac{\alpha}{r} \Delta W = W + \frac{\alpha}{r} AB$$

(r : rank, α : scaling factor)



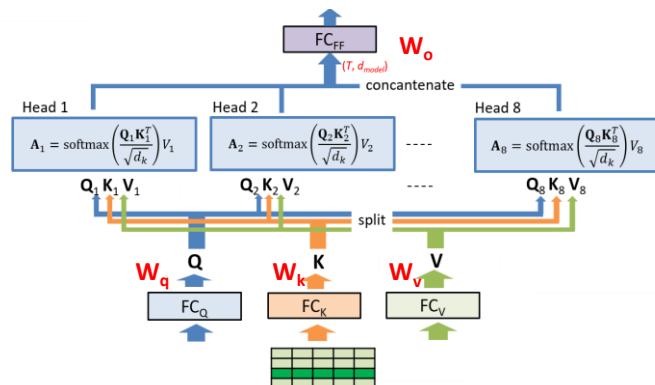
- Low-rank matrices have fewer parameters.

$$\begin{aligned}\text{\#params in } \Delta W &= d * k \\ &= 4 * 8 \\ &= 32\end{aligned}$$

$$\begin{aligned}\text{\#params in } AB &= d * r + r * k \\ &= 4 * 2 + 2 * 8 \\ &= 24\end{aligned}$$

LoRA: which weight matrices should be adapted?

- There are 4 FC layers in the self-attention layer. How does adaptation on their weights affect the performance?



- Adapting **all** layers give the best performance

	# of Trainable Parameters = 18M						
Weight Type	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v	W_q, W_k, W_v, W_o
Rank r	8	8	8	8	4	4	2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

Table: Validation accuracy on WikiSQL and MultiNLI after applying LoRA to different attention weights in GPT-3

LoRA: what is the optimal rank r for LoRA?

- LoRA performs competitively with **small r** especially for $\{W_q, W_v\}$
- ΔW could have a very small intrinsic rank
- Microsoft LoRA uses $r = 8$

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

Table: Validation accuracy on WikiSQL and MultiNLI on GPT-2 with different rank r

LoRA: Performance on GPT-3 175B

- LoRA ($r=8$) outperforms full fine-tuning
- Outperforms other adaptation methods at the time

Model&Method		# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
			Acc. (%)	Acc. (%)	R1/R2/RL
full-finetuning	GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
	GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
Other adaptation methods	GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
	GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
	GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
	GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
LoRA ($r=1$)	GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
LoRA ($r=8$)	GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

Table: Performance on different adaptation methods on GPT-3 175B

Benefits of LoRA

- Finetuning is very **efficient** because of small number of trainable parameters
- Multiple lightweight and portable LoRA models can **share** a single base model.
- Orthogonal to other parameter-efficient methods and can be **combined** with many of them.
- Performance is **comparable** to state-of-the-art method.

*Thank you
for your attention*